

Database Architecture

CSI 2004
Advance Database Management
Systems

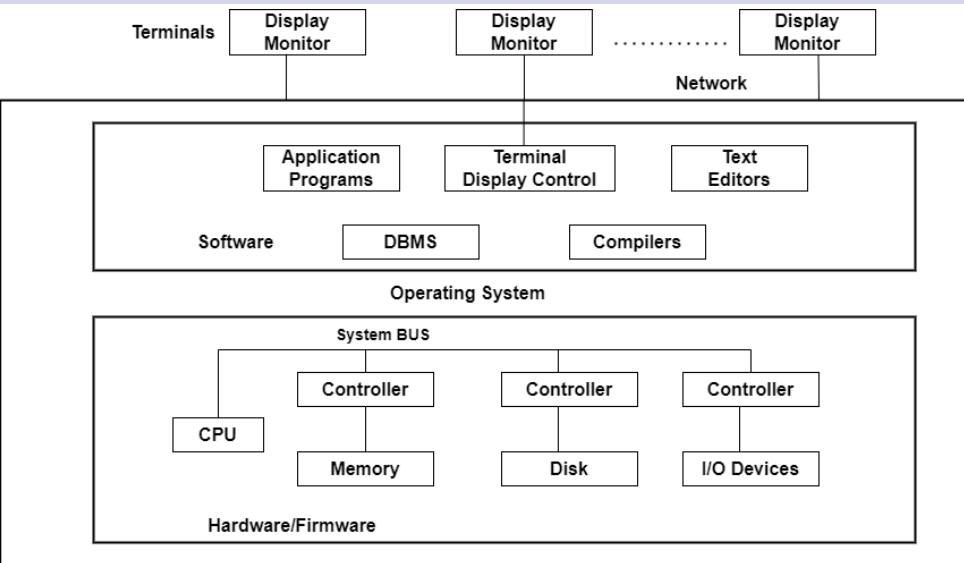
Centralized and Client Server Architecture for DBMS

- Architectures for DBMSs have generally followed trends seen in architectures for larger computer systems.
- The primary processing for all system functions, including user application programs, user interface programs, and all DBMS capabilities, was handled by mainframe computers in earlier systems.
- The primary cause of this was that the majority of users accessed such systems using computer terminals with limited processing power and merely display capabilities.
- Only display data and controls were delivered from the computer system to the display terminals, which were connected to the central node by a variety of communications networks, while all processing was done remotely on the computer system.

Centralized and Client Server Architecture for DBMS

- The majority of users switched from terminals to PCs and workstations as hardware prices decreased.
- Initially, Database Systems operated on these computers in a manner akin to how they had operated display terminals.
- As a result, the DBMS itself continued to operate as a centralized DBMS, where all DBMS functionality, application program execution, and UI processing were done on a single computer.
- The physical elements of a centralized architecture Client/server DBMS designs emerged as DBMS systems gradually began to take advantage of the user side's computing capability.

Centralized and Client Server Architecture for DBMS



Centralized architecture of DBMS

Client Server Architecture for DBMS

- We first talk about client/server architecture in general, and then we look at how DBMSs use it.
- In order to handle computing settings with a high number of PCs, workstations, file servers, printers, database servers, etc., the client/server architecture was designed.
- A network connects various pieces of software and hardware, including email and web server software.
- To define specialized servers with a particular functionality is the aim.
- For instance, it is feasible to link a number of PCs or compact workstations to a file server that manages the client machines' files as clients.
- By having connections to numerous printers, different devices can be designated as a printer server; all print requests from clients are then directed to this machine.

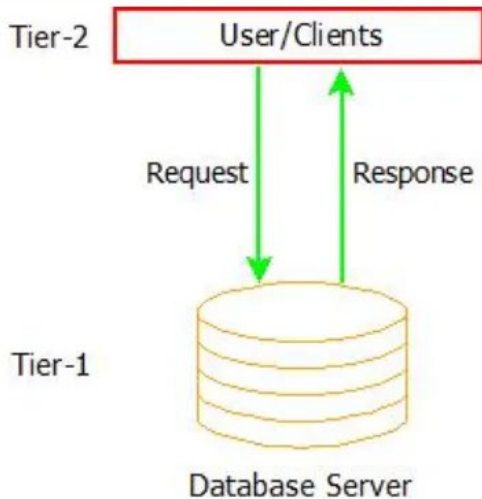
Client Server Architecture for DBMS

- The category of specialized servers also includes web servers and email servers.
- Many client machines can utilize the resources offered by specialized servers.
- The user is given the proper user interfaces for these servers as well as local processing power to run local applications on the client devices.
- This idea can be applied to various types of software, where specialist applications, like a CAD (computer-aided design) package, are kept on particular server computers and made available to a variety of clients.
- Some devices (such as workstations or PCs with discs that only have client software installed) would only be client sites.

Two Tier Client Server Architecture for DBMS

- Two-tier means that our Architecture has two layers, which are client layer and Data layer.
- In Client layer we have several Client machines which can have the access to the database server.
- The API present on the client machine will establish the connection between the machine and the Database server through JDBC something else.
- This is because Clients and Database Server may be at different locations.
- Once this connection gets established, the Interface present on the client machine contains an Application Program on the back-side which contains a query.
- This query will be processed by the Database server and in turn the queried information will be sent to the client machine.

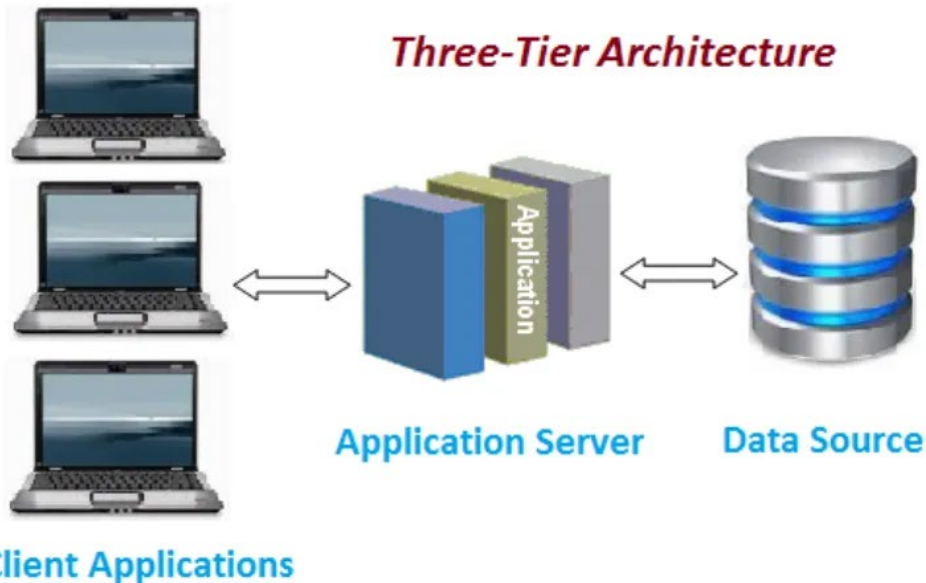
Two Tier Client Server Architecture for DBMS



Three Tier Client Server Architecture for DBMS

- Here there is an additional layer which acts as an intermediate between Client layer and Datalayer called Business logic layer.
- Business logic layer is the layer where the Application Programs are processed.
- Here the Application Programs are processed in the Application server itself, which makes it different from Two-tier Architecture where queries are processed in the database server.
- Simply the Client machines will contact Application Server which in turn processes our Application Programs and fetches the Required Data from Database and then sends this Information back to the client machine in the suitable format only.

Three Tier Client Server Architecture for DBMS



Parallel Database

- Parallel DBMS is a Database Management System that runs through multiple processors and disks.
- They combine two or more processors also disk storage that helps make operations and executions easier and faster.
- They are designed to execute concurrent operations.
- They exist, happen, or done at the same time even if the data processed are not from one source or one processing unit.

Architecture for Parallel Database

- **Shared Memory System**

- A Shared Memory System is an architecture of Database Management System, where every computer processor is able to access and process data from multiple memory modules or unit through intercommunication channel.
- This architecture is also commonly known as SMP or Symmetric Multi-processing. A Shared Memory System contains large amounts of cache memories at each processor, so referencing of the shared memory is avoided.

- **Advantages of Shared Memory:**

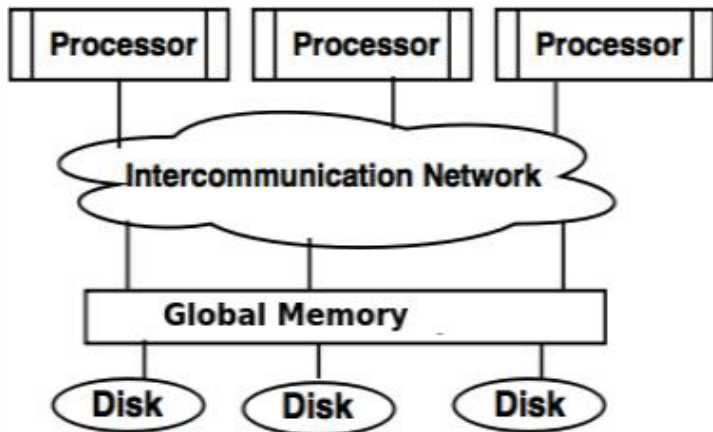
- Data is easily accessed from various processors.
- A single processor can send messages to other processors efficiently.

- **Disadvantage of Shared Memory:**

- Waiting time for every single processor increases when all of them are uses.
- Bandwidth is also a problem.

Architecture for Parallel Database

- Shared Memory System



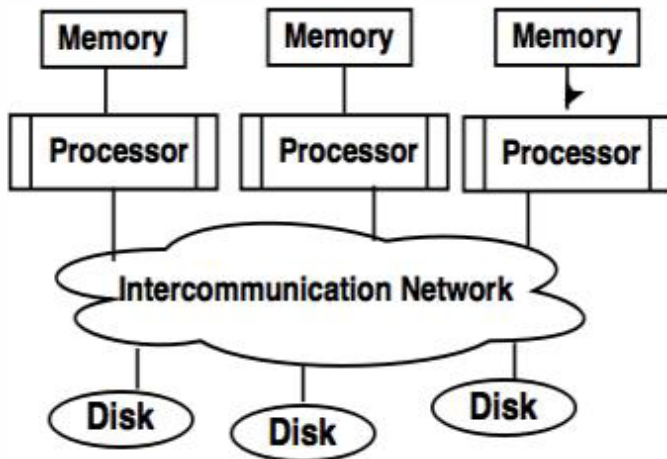
Shared Memory System in Parallel Databases

Architecture for Parallel Database

- **Shared Disk System**
 - A Shared Disk System is an architecture of Database Management System where every computer processors can access multiple disk through intercommunication network. It can also access and utilize every local memory. Each of the processors have own memory system, so the shared data are more efficient.
- **Advantages of Shared Disk System:**
 - The fault tolerance can be achieved using this system.
- **Disadvantages of Shared Disk System:**
 - The addition of processors can slow down existing processors.
 - Shared Disk System have a limited scalability meaning it is sometimes fixed.

Architecture for Parallel Database

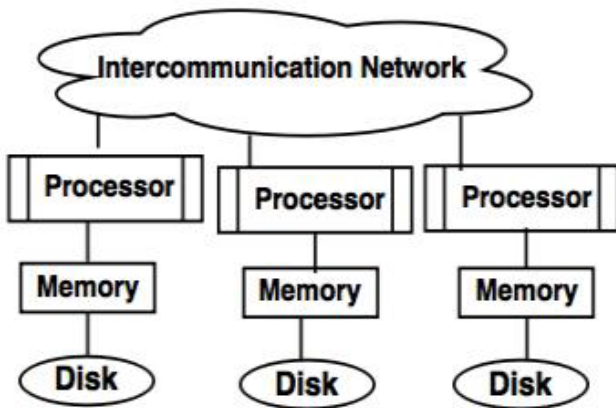
- Shared Disk System



Shared disk system in Parallel Databases

Architecture for Parallel Database

- Shared Nothing System



Shared nothing disk system in Parallel Databases

Distributed Database System

- A Distributed Database System is a kind of database that is present or divided in more than one location, which means it is not limited to any single computer system.
- It is divided over the network of various systems.
- The Distributed Database System is physically present on the different systems in different locations.
- This can be necessary when different users from all over the world need to access a specific database.
- For a user, it should be handled in such a way that it seems like a single database.

Parameters of Distributed Database Systems

- **Distribution:**
 - It describes how data is physically distributed among the several sites.
- **Autonomy:**
 - It reveals the division of power inside the Database System and the degree of autonomy enjoyed by each individual DBMS.
- **Heterogeneity:**
 - It speaks of the similarity or differences between the databases, system parts, and data models.

Architecture Models of Database Systems

Client-Server Architecture of DDBMS:

- This architecture is two level architecture where clients and servers are the points or levels where the main functionality is divided. There is various functionality provided by the server, like managing the transaction, managing the data, processing the queries, and optimization.

Peer-to-peer Architecture of DDBMS:

- In this architecture, each node or peer is considered as a server as well as a client, and it performs its database services as both (server and client). The peers coordinate their efforts and share their resources with one another.

Multi DBMS Architecture of DDBMS:

- This is an amalgam of two or more independent Database Systems that functions as a single integrated Database System.

Types of Distributed Database Systems

- **Homogeneous Database System:**

- Each site stores the same database in a Homogenous Database. Since each site has the same database stored, so all the data management schemes, operating system, and data structures will be the same across all sites.
- They are, therefore, simple to handle.

- **Heterogeneous Database System:**

- In this type of Database System, different sites are used to store the data and relational tables, which makes it difficult for database administrators to do the transactions and run the queries into the database.
- Additionally, one site might not even be aware of the existence of the other sites. Different operating systems and database applications may be used by various computers.
- Since each system has its own database model to store the data, therefore it is required there should be translation schemes to establish the connections between different sites to transfer the data.

Distributed Data Storage

- There are two methods by which we can store the data on different sites:
- **Replication:**
 - This method involves redundantly storing the full relationship at two or more locations. Since a complete database can be accessed from each site, it becomes a redundant database. Systems preserve copies of the data as a result of replication.
 - This has advantages because it makes more data accessible at many locations. Additionally, query requests can now be handled in parallel.
 - However, there are some drawbacks as well. Data must be updated frequently. Any changes performed at one site must be documented at every site where that relation is stored in order to avoid inconsistent results. There is a tonne of overhead here. Additionally, since concurrent access must now be monitored across several sites, concurrency control becomes far more complicated.

Distributed Data Storage

- **Fragmentation:**

- According to this method, the relationships are divided (i.e., broken up into smaller pieces), and each fragment is stored at the many locations where it is needed. To ensure there is no data loss, the pieces must be created in a way that allows for the reconstruction of the original relation.
- Since Fragmentation doesn't result in duplicate data, consistency is not a concern.

- Horizontal Fragmentation:

- In Horizontal Fragmentation, the relational table or schema is broken down into a group of one and more rows, and each row gets one fragment of the schema. It is also called splitting by rows.

- Vertical Fragmentation:

- In this fragmentation, a relational table or schema is divided into some more schemas of smaller sizes. A common candidate key must be present in each fragment in order to guarantee a lossless join. This is also called splitting by columns.

Parameters of Distributed Database Systems

- Distribution:
- It describes how data is physically distributed among the several sites.
- Autonomy:
- It reveals the division of power inside the Database System and the degree of autonomy enjoyed by each individual DBMS.
- Heterogeneity:
- It speaks of the similarity or differences between the databases, system parts, and data models.

Application of Distributed Database Systems

- Multimedia apps use it.
- The manufacturing control system also makes use of it.
- Another application is by corporate management for the information system.
- It is used in hotel chains, military command systems, etc.

Physical Database Design and Tuning

CSI 2004
Advance Database Management
Systems

Outline

- ① Introduction
- ② Designing and Tuning the Physical Schema
 - Indexing
 - Guidelines for Physical Design
- ③ Tuning the Conceptual Schema
 - Denormalization
 - Partitioning
- ④ Tuning Queries and Applications

Physical Database Design and Tuning

Physical Design The process of selecting a physical schema (collection of data structures) to implement the conceptual schema

Tuning Periodically adjusting the physical and/or conceptual schema of a working system to adapt to changing requirements and/or performance characteristics

Good design and tuning requires understanding the database **workload**.

Workload Modeling

Definition (Workload Description)

A *workload description* contains

- the most important queries and their frequency
 - the most important updates and their frequency
 - the desired performance goal for each query or update
-
- For each query:
 - Which relations are accessed?
 - Which attributes are retrieved?
 - Which attributes occur in selection/join conditions? How *selective* is each condition?
 - For each update:
 - Type of update and relations/attributes affected.
 - Which attributes occur in selection/join conditions? How *selective* is each condition?

The Physical Schema

- A storage strategy is chosen for each relation
 - Possible storage options:
 - Unsorted (heap) file
 - Sorted file
 - Hash file
- Indexes are then added
 - Speed up queries
 - Extra update overhead
 - Possible index types:
 - B-trees (actually, B+-trees)
 - R trees
 - Hash tables
 - ISAM, VSAM
 - ...

A Table Scan

```
select *  
from Employee  
where Lastname = 'Smith'
```

- To answer this query, the DBMS must search the blocks of the database file to check for matching tuples.
- If no indexes exist for Lastname (and the file is unsorted with respect to Lastname), all blocks of the file must be scanned.

Creating Indexes

```
create index LastnameIndex  
on Employee(Lastname) [CLUSTER]
```

```
drop index LastnameIndex
```

Primary effects of LastnameIndex:

- Substantially reduce execution time for selections that specify conditions involving Lastname
- Increase execution time for insertions
- Increase or decrease execution time for updates or deletions of tuples from Employee
- Increase the amount of space required to represent Employee

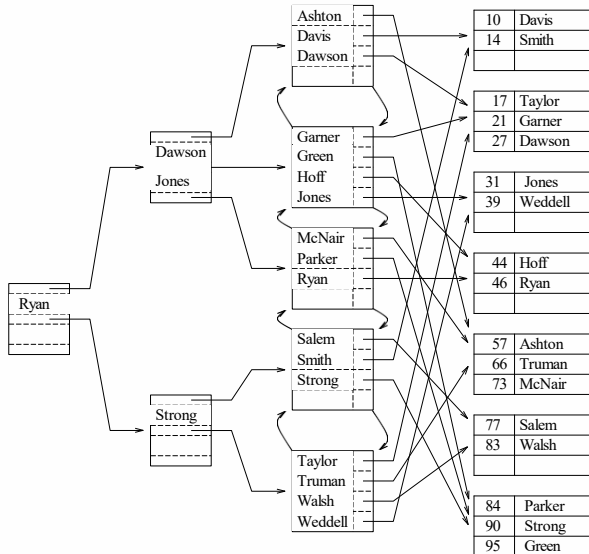
Clustering vs. Non-Clustering Indexes

- An index on attribute A of a relation is a **clustering** index if tuples in the relation with similar values for A are stored together in the same block.
- Other indices are **non-clustering** (or secondary) indices.

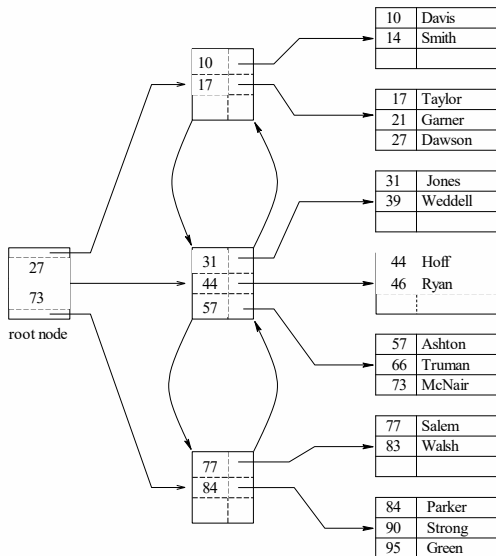
Note

A relation may have at most one clustering index, and any number of non-clustering indices.

Non-Clustering Index Example



Clustering Index Example



Co-Clustering Relations

Definition (Co-Clustering)

Two relations are **co-clustered** if their tuples are interleaved within the same file

- Co-clustering is useful for storing hierarchical data (1:N relationships)
- Effects on performance:
 - Can speed up joins, particularly foreign-key joins
 - Sequential scans of either relation become slower

Range Queries

- B-trees can also help for **range queries**:

```
select *  
from R  
where A > c
```

- If a B-tree is defined on A , we can use it to find the tuples for which $A \leq c$. Using the forward pointers in the leaf blocks, we can then find tuples for which $A > c$.

Multi-Attribute Indices

- It is possible to create an index on several attributes of the same relation. For example:

```
create index NameIndex  
on Employee (Lastname,Firstname)
```

- The order in which the attributes appear is important. In this index, tuples (or tuple pointers) are organized first by Lastname. Tuples with a common surname are then organized by Firstname.

Using Multi-Attribute Indices

- The NameIndex index would be useful for these queries:

```
select *  
from Employee  
where Lastname = 'Smith'
```

```
select *  
from Employee  
where Lastname = 'Smith'  
and Firstname = 'John'
```

- It would be *very* useful for these queries:

```
select Firstname  
from Employee  
where Lastname = 'Smith'
```

```
select Firstname, Lastname  
from Employee
```

- It would not be useful at all for this query:

```
select *  
from Employee  
where Firstname = 'John'
```

Physical Design Guidelines

- ① Don't index unless the performance increase outweighs the update overhead
- ② Attributes mentioned in WHERE clauses are candidates for index search keys
- ③ Multi-attribute search keys should be considered when
 - a WHERE clause contains several conditions; or
 - it enables index-only plans
- ④ Choose indexes that benefit as many queries as possible
- ⑤ Each relation can have at most one clustering scheme; therefore choose it wisely
 - Target important queries that would benefit the most
 - Range queries benefit the most from clustering
 - Join queries benefit the most from co-clustering
 - A multi-attribute index that enables an index-only plan does not benefit from being clustered

DB2 Index Advisor

```
% db2advis -d sample -s "select empno,lastname
from employee where workdept = 'xxxx'"
Found maximum set of [1] recommended indexes
total disk space needed for initial set [ 0.005] MB
[ 50.5219] timerons (without indexes)
[ 25.1521] timerons (with current solution)
[%50.22] improvement
-- =====
-- index[1],      0.005MB
CREATE INDEX WIZ1517 ON "KMSALEM"."EMPLOYEE"
("WORKDEPT" ASC, "LASTNAME" ASC, "EMPNO" ASC) ;
-- =====
```


Tuning the Conceptual Schema

Suppose that after tuning the physical schema, the system still does not meet the performance goals!

- Adjustments can be made to the conceptual schema:
 - Re-normalization
 - Denormalization
 - Partitioning

Warning

Unlike changes to the physical schema, changes to the conceptual schema of an operational system—called *schema evolution*—often can't be completely masked from end users and their applications.

Denormalization

Normalization is the process of decomposing schemas to reduce redundancy

Denormalization is the process of merging schemas to intentionally *increase* redundancy

In general, redundancy *increases update overhead* (due to change anomalies) but *decreases query overhead*.

The appropriate choice of normal form depends heavily upon the workload.

Partitioning

- Very large tables can be a source of performance bottlenecks
- *Partitioning* a table means splitting it into multiple tables for the purpose of reducing I/O cost or lock contention

① Horizontal Partitioning

- Each partition has all the original columns and a subset of the original rows
- Tuples are assigned to a partition based upon a (usually natural) criteria
- Often used to separate operational from archival data

② Vertical Partitioning

- Each partition has a subset of the original columns and all the original rows
- Typically used to separate frequently-used columns from each other (concurrency *hot-spots*) or from infrequently-used columns

Tuning Queries

- Changes to the physical or conceptual schemas impacts *all* queries and updates in the workload.
- Sometimes desirable to target performance of specific queries or applications
- Guidelines for tuning queries:
 - ① Sorting is expensive. Avoid unnecessary uses of ORDER BY, DISTINCT, or GROUP BY.
 - ② Whenever possible, replace subqueries with joins
 - ③ Whenever possible, replace correlated subqueries with uncorrelated subqueries
 - ④ Use vendor-supplied tools to examine generated plan. Update and/or create statistics if poor plan is due to poor cost estimation.

Tuning Applications

Guidelines for tuning applications:

- ① Minimize communication costs
 - Return the fewest columns and rows necessary
 - Update multiple rows with a WHERE clause rather than a cursor
- ② Minimize lock contention and hot-spots
 - Delay updates as long as possible
 - Delay operations on hot-spots as long as possible
 - Shorten or split transactions as much as possible
 - Perform insertions/updates/deletions in batches
 - Consider lower isolation levels